

AI-powered Drone Delivery Management Platform

Group 4

089206011070 - Tran Tan Phat

079206000967 - Tran Hoang Son

056206000983 - Nguyen Duy Quy

054206007389 - Nguyen Quoc Dung

079206004792 - Le Huy Hoang

September 16, 2026

Contents

PREFACE	iii
INTRODUCTION	iv
0.1 SmartDroneDelivery Platform	iv
0.2 Database Perspective	v
0.3 Scope of This Document	vi
0.4 Document Organization	vi
DATABASE REQUIREMENTS ANALYSIS	vii
0.5 System Overview	vii
0.6 System Actors and Data Responsibilities	vii
0.7 Functional Requirements	ix
0.8 Non-functional Requirements	xi
0.9 Business Process	xii
0.10 Business Rules	xiii
0.11 Data Requirements	xvi
0.12 Requirement-to-Entity Traceability	xvii
0.13 Chapter Summary	xvii
DATABASE DESIGN	xix
0.14 Conceptual Database Design	xix
0.15 Logical Database Design	xxix
0.16 Physical Database Design	xxxiii
0.17 Chapter Summary	xliv
DATABASE TESTING AND EVALUATION	xlvi
0.18 Testing Objectives	xlvi
0.19 Constraint Tests	xlvi
0.20 Business Rule Tests	xlvi
0.21 History Preservation Tests	l
0.22 Audit Log Verification Test	li
0.23 Query and Performance Tests	li

0.24	Chapter Summary	li
CONCLUSION		liii
0.25	Database Design Results	liii
0.26	Limitations	liv
0.27	Future Database Improvements	lv

Preface

The rapid development of e-commerce and smart logistics has created an increasing demand for efficient and intelligent last-mile delivery solutions. Drone delivery has emerged as a promising approach for transporting lightweight packages in environments such as smart campuses, industrial parks, residential communities, and remote areas.

The SmartDroneDelivery project is proposed as an AI-powered Drone Delivery Management Platform that focuses on managing drone-based delivery operations rather than directly controlling drones. The platform is designed to support the complete delivery lifecycle, including delivery request management, package management, delivery execution, delivery tracking, landing station management, and delivery confirmation. In addition, AI-assisted services are introduced to support estimated delivery time, delivery summarization, customer assistance, and operational analytics.

As an important component of the overall system, the database is responsible for organizing and maintaining the data generated throughout these business processes. A well-designed database is essential for ensuring data consistency, integrity, traceability, security, and efficient access for the application layer.

This document focuses specifically on the database design of the SmartDroneDelivery platform. The scope includes requirement analysis related to data, identification of entities and relationships, conceptual and logical database design, relational schema design, integrity constraints, indexing, and database implementation using PostgreSQL.

Introduction

The rapid growth of e-commerce and smart logistics has increased the demand for efficient last-mile delivery solutions. Drone delivery has emerged as a promising method for transporting lightweight packages in environments such as smart campuses, industrial parks, residential areas, and remote locations. However, effective drone delivery requires more than drone operation alone. It involves managing customers, delivery requests, packages, landing stations, delivery tracking, and confirmation through a centralized and reliable information system. Existing logistics systems are mainly designed for traditional transportation and often lack dedicated support for drone-based workflows. To address these challenges, the SmartDroneDelivery project proposes an AI-powered Drone Delivery Management Platform that enables efficient management, coordination, and monitoring of drone delivery operations in a business environment.

0.1. SmartDroneDelivery Platform

SmartDroneDelivery is designed to support the complete delivery lifecycle, from the creation of a delivery request to the confirmation of successful delivery. The platform focuses on business and logistics management rather than direct drone control.

The system supports five primary business processes:

1. Delivery request management
2. Package management
3. Delivery execution
4. Delivery tracking
5. Delivery confirmation

In addition to these core processes, AI-assisted services are proposed to improve operational efficiency and customer experience. These services include estimated delivery time,

delivery summarization, customer assistance, and operational analytics. The platform supports five main human actors:

1. Customer
2. Dispatcher
3. Station Operator
4. Logistics Manager
5. System Administrator

The system is organized into five core modules:

1. Customer and Delivery Order Management
2. Package and Delivery Workflow Management
3. Landing Station Management
4. Delivery Tracking and Confirmation
5. Analytics and AI-assisted Services

The proposed platform provides a web portal for logistics management, a mobile application for customers and operators, and standardized RESTful APIs for integration with external services.

0.2. Database Perspective

Within the SmartDroneDelivery platform, the database plays a central role in storing and managing information generated by the different business processes. The database must support the management of users, customers, addresses, delivery orders, packages, drones, landing stations, delivery activities, tracking information, delivery confirmation, and other supporting information required by the system.

The database design must ensure that data is organized in a structured manner and that relationships among different entities are clearly defined. It must also maintain data integrity and consistency through appropriate primary keys, foreign keys, constraints, and other database mechanisms.

A well-designed database provides a reliable data foundation for the backend services and allows the frontend and other system components to access and process data consistently. Therefore, database design is an essential part of the SmartDroneDelivery platform.

0.3. Scope of This Document

This document focuses specifically on the database design of the SmartDroneDelivery platform. The main objectives of the database design are:

1. Analyze the data requirements derived from the proposed business processes.
2. Identify the main entities and their attributes.
3. Define relationships and cardinalities among entities.
4. Develop the conceptual database model.
5. Transform the conceptual model into a logical relational schema.
6. Apply database normalization principles to reduce redundancy and improve consistency.
7. Define primary keys, foreign keys, and integrity constraints.
8. Design indexes and other database structures to support efficient data access.
9. Implement the database using PostgreSQL.
10. Provide a consistent database foundation for future backend and frontend integration.

The database design presented in this document is therefore intended to serve as the data foundation of the SmartDroneDelivery platform and to support the future implementation of its web, mobile, API, and AI-assisted services.

0.4. Document Organization

The document is organized into several sections covering the analysis and design of the SmartDroneDelivery database. The introductory section presents the project background, system overview, database perspective, and scope of the database design. The following sections describe the data requirements and business rules, followed by the conceptual database model, logical relational schema, normalization, integrity constraints, physical database design, implementation, and testing. Finally, the document presents the database implementation results and provides conclusions regarding the proposed database design.

Database Requirements Analysis

0.5. System Overview

The SmartDroneDelivery platform is an AI-powered Drone Delivery Management Platform designed to support the management of drone-based delivery operations. The system focuses on business and logistics management rather than direct drone control.

The platform supports the complete delivery lifecycle, beginning with the creation of a delivery request and continuing through package management, delivery scheduling, delivery execution, delivery tracking, and delivery confirmation.

From a database perspective, the system requires a structured and reliable data management layer capable of storing and maintaining all information generated throughout these business processes. This chapter identifies the actors, functional requirements, business rules, and data requirements that form the foundation of the database design.

0.6. System Actors and Data Responsibilities

The SmartDroneDelivery platform supports five main human actors. Each actor interacts with different functions of the system and is associated with specific categories of data that the database must manage.

0.6.1. Customer

The Customer actor represents individuals who use the platform to request drone delivery services. The database must manage the following data on behalf of this actor:

- Customer identification and profile information.
- Customer account linked to the system user record.
- One or more delivery addresses associated with the customer.

- Delivery orders created by the customer.
- Delivery confirmation records associated with completed orders.

0.6.2. Dispatcher

The Dispatcher actor is a staff member responsible for reviewing and scheduling delivery orders. The database must provide this actor with access to:

- Pending delivery orders awaiting review.
- Current order status and status history.
- Delivery activity records, including drone assignment.
- Rescheduling information for failed or cancelled deliveries.

0.6.3. Station Operator

The Station Operator actor manages the physical landing stations used in delivery operations. The relevant data includes:

- Landing station identification and location information.
- Current station status and capacity.
- Station status history records.
- Station operator assignments linking operators to designated stations.
- Package station events, including arrival, pickup confirmation, loading, and unloading.
- Delivery activities originating from or terminating at a given station.

0.6.4. Logistics Manager

The Logistics Manager actor monitors overall operations and uses data for reporting and analytics. The relevant data includes:

- Aggregated delivery order and activity data.
- Tracking records for monitoring delivery progress and flight trajectories.
- Delivery exception, incident, and failure records.
- Station operational metrics and operator assignments.
- Data required for operational reports and AI-assisted analytics.

0.6.5. System Administrator

The System Administrator actor manages the technical aspects of the system. The relevant data includes:

- System user accounts and account status.
- Role and permission assignments.
- Audit log records for all important system actions.

0.7. Functional Requirements

This section describes the main functions that the SmartDroneDelivery platform must support from a database perspective. The functional requirements are grouped according to the core modules of the system.

0.7.1. User Management

The user management module handles account registration, authentication, and role-based access control for all system actors.

1. The system shall allow customers to register and manage their own accounts.
2. The system shall allow administrators to create, update, and deactivate accounts for staff members, including dispatchers, station operators, and logistics managers.
3. The system shall authenticate users through a secure login mechanism using password hashing.
4. The system shall assign roles and permissions to each account and restrict access to functions according to that role.
5. The system shall allow users to view and update their personal profile information.

This module requires the database to store account credentials (`system_user`), roles (`role`), permissions (`permission`), and their bridge assignments (`user_role`, `role_permission`).

0.7.2. Customer and Delivery Order Management

This module supports the management of customer information and delivery orders.

1. The system shall allow customers to manage one or more delivery addresses, including setting a default address and geographic coordinates.
2. The system shall allow customers to create a delivery request by specifying the delivery address, requested delivery time, and package physical details.

3. The system shall allow dispatchers to review, approve, or reject submitted delivery requests.
4. The system shall allow customers and staff to view the current status and full history of their delivery orders.
5. The system shall allow a delivery order to be modified or cancelled before it is scheduled.

This module requires the database to store customer profiles (customer), delivery addresses (customer_address), delivery orders (delivery_order), package details (package), and order status history (order_status_history).

0.7.3. Delivery Workflow and Execution

This module supports the scheduling, execution, and confirmation of delivery activities.

1. The system shall allow dispatchers to schedule an approved delivery order by creating a delivery activity, selecting pickup and destination stations, and assigning an available operational drone.
2. The system shall verify that the total package weight does not exceed the assigned drone's payload capacity.
3. The system shall record package station events, ensuring package pickup and loading are confirmed by an assigned station operator prior to drone departure.
4. The system shall record real-time flight telemetry and tracking information generated during delivery execution.
5. The system shall allow confirmation of delivery upon successful handover, capturing receiver details, confirmation method, and a digital evidence key.
6. The system shall record failed delivery attempts with comprehensive exception reasons and support rescheduling by creating a new linked delivery activity.

This module requires the database to store delivery activities (delivery_activity), package station events (package_station_event), tracking records (tracking_record), delivery confirmations (delivery_confirmation), and delivery exceptions (delivery_exception).

0.7.4. Landing Station and Operator Management

This module supports the management of landing stations and station staffing used during delivery operations.

1. The system shall allow station operators to register and update landing station information, including geographic coordinates and capacity.

2. The system shall record operator assignments to stations, ensuring only assigned operators perform station operations.
3. The system shall monitor station availability and record all station status changes, such as maintenance or capacity limits, in an immutable history.

This module requires the database to store landing stations (`landing_station`), station status history (`station_status_history`), and station operator assignments (`station_operator_assignment`).

0.7.5. Dashboard and AI-assisted Services

This module provides operational monitoring and AI-assisted functions to support logistics managers and dispatchers.

1. The system shall provide a dashboard displaying the status of delivery orders, drones, and landing stations.
2. The system shall store AI-generated recommendations for drone selection, route planning, and estimated time of arrival (ETA).
3. The system shall provide operational analytics based on historical delivery and tracking data.

This module requires the database to store historical operational data, planned routes (`delivery_route`, `route_point`), and AI recommendation results (`ai_recommendation`). The database does not store AI model weights or code.

0.8. Non-functional Requirements

In addition to functional requirements, the system must satisfy several non-functional requirements that directly affect database design decisions.

0.8.1. Security

The system shall provide secure authentication and role-based authorization. Sensitive credentials must be stored as cryptographic hashes (`password_hash`). The database design must enforce access control through relational permissions and prevent unauthorized access to customer and operational records.

0.8.2. Performance

The system is expected to provide a response time of less than three seconds for common operational queries. Selective indexing, optimal data types, and non-redundant schemas must be implemented to support high-throughput telemetry writes and rapid lookup.

0.8.3. Concurrency

The system shall support concurrent users, dispatchers, and automated telemetry ingestion while maintaining strict data consistency. Transactions and constraints must prevent race conditions, such as double-booking a drone for overlapping flights.

0.8.4. Integration

The database serves as the persistent foundation for RESTful APIs and event-driven services. Object storage keys (`evidence_object_key`) are stored to link external object storage (MinIO/S3) without bloating PostgreSQL with binary files.

0.8.5. Scalability and Maintainability

The database design must be modular, adhering to Third Normal Form (3NF), allowing future functional extensions (e.g., maintenance schedules, multi-hop routes) without breaking existing relational schemas.

0.8.6. Audit and Traceability

The system shall maintain comprehensive auditability. An append-only audit log table must capture the actor, action type, entity, changed values, and timestamp for all critical data modifications.

0.9. Business Process

The complete delivery lifecycle is managed through the following systematic workflow:

1. **Order Placement:** A customer creates a delivery order specifying the target delivery address, requested delivery time, and package specifications. The order is stored with an initial status of PENDING.
2. **AI Recommendation:** The AI engine processes the order requirements and records recommendations for drone selection and ETA prediction in `ai_recommendation`.
3. **Review and Approval:** The dispatcher reviews the order and AI recommendations, updating the order status to APPROVED or REJECTED.
4. **Scheduling and Drone Assignment:** The dispatcher schedules the delivery, creating a `delivery_activity` that links the order to a specific drone, departure station, and destination station. The order status transitions to ASSIGNED.

5. **Station Processing and Package Pickup:** The package arrives at the departure station. An assigned station operator inspects the package, recording a `package_station_event` with status `PICKUP_CONFIRMED` and `LOADED`.
6. **Flight Execution and Tracking:** The drone takes off. The activity transitions to `IN_TRANSIT`. High-frequency telemetry records (coordinates, altitude, speed, battery) are continuously inserted into `tracking_record`.
7. **Successful Delivery Confirmation:** Upon safe arrival and recipient handover, the recipient signs or provides verification. A `delivery_confirmation` record is created with a reference to the proof object in storage. The order and activity statuses are updated to `DELIVERED`.
8. **Exception and Rescheduling:** If flight anomalies or weather issues occur, a `delivery_exception` is recorded, and the activity status is updated to `FAILED`. The dispatcher can reschedule the delivery, which creates a new `delivery_activity` linked to the original failed activity via `previous_activity_id`, preserving complete historical traceability.

0.10. Business Rules

To ensure complete structural rigor, thirty-nine formal business rules (BR-01 through BR-39) are established. These rules directly govern entity identification, relationship cardinalities, and database integrity constraints.

Customer and Address Rules.

- **BR-01:** A customer can have multiple delivery addresses.
- **BR-02:** Each delivery order belongs to exactly one customer.
- **BR-03:** Each delivery order uses exactly one delivery address.
- **BR-04:** The selected delivery address must belong to the customer who owns the order.
- **BR-05:** A customer can have at most one default delivery address.

Order and Package Rules.

- **BR-06:** A delivery order must contain at least one package.
- **BR-07:** A delivery order can contain multiple packages.
- **BR-08:** Package weight and physical dimensions (length, width, height) must be strictly positive numeric values.

- **BR-09:** Package status must follow the defined lifecycle (CREATED, AT_STATION, PICKED_UP, IN_TRANSIT, DELIVERED, FAILED, RETURNED, DAMAGED).

Order Status and Lifecycle Rules.

- **BR-10:** Every delivery order has exactly one current status at any given moment.
- **BR-11:** Every order status change must be preserved in an append-only `order_status_history` table.
- **BR-12:** Only valid status transitions are permitted in the order lifecycle.
- **BR-13:** A cancelled order cannot return to an active state.
- **BR-14:** A delivered order cannot be rescheduled.

Drone and Operational Rules.

- **BR-15:** A drone can execute many delivery activities over time.
- **BR-16:** A drone cannot be assigned to two active delivery activities simultaneously.
- **BR-17:** A drone assigned to an active delivery must be in an operational status (AVAILABLE).
- **BR-18:** The total weight of packages assigned to an activity must not exceed the drone's payload capacity (`max_payload_kg`).

Delivery Activity and Rescheduling Rules.

- **BR-19:** One delivery order may be associated with multiple delivery activities over time.
- **BR-20:** Each delivery activity represents one discrete delivery execution attempt.
- **BR-21:** Failed delivery attempt records, exceptions, and tracking data must be preserved.
- **BR-22:** Rescheduling an order creates a new delivery activity that references the failed activity via `previous_activity_id`.
- **BR-23:** Each delivery activity is associated with exactly one drone.
- **BR-24:** Each delivery activity has its own independent status and operational timestamps.

Landing Station and Staffing Rules.

- **BR-25:** A landing station has a defined non-negative capacity.

- **BR-26:** Station status transitions (e.g., maintenance, offline) must be preserved in `station_status_history`.
- **BR-27:** A station operator can be assigned to one or more landing stations over specified time intervals.
- **BR-28:** Only actively assigned station operators may record station operations and verify package pickups.

Package Pickup Rules.

- **BR-29:** A package can be processed at a departure or destination landing station.
- **BR-30:** Package pickup and handover must be explicitly recorded as station events.
- **BR-31:** Package station event records must record the station, performing operator, event type, and timestamp.
- **BR-32:** A package cannot transition to `IN_TRANSIT` without a verified pickup event.

Delivery Confirmation Rules.

- **BR-33:** A delivery order has at most one successful delivery confirmation.
- **BR-34:** A delivery confirmation can only be generated for a successfully delivered order.
- **BR-35:** A confirmation is linked directly to the specific successful delivery activity.
- **BR-36:** Recipient identification, confirmation method, and digital proof reference must be preserved.

Audit and Traceability Rules.

- **BR-37:** All critical system actions must be traceable to the responsible user and timestamp.
- **BR-38:** Audit records must capture actor identity, action type, affected table, record ID, and before/after state.
- **BR-39:** Business state history (`order_status_history`, `station_status_history`) and audit logs (`audit_log`) represent separate concerns and must be stored in distinct tables.

0.11. Data Requirements

Based on the system requirements and thirty-nine business rules, the database must manage the following major categories of data.

0.11.1. User and Authorization Data

Stores authentication credentials, account statuses, roles, permissions, and many-to-many role/permission assignments (`system_user`, `role`, `permission`, `user_role`, `role_permission`).

0.11.2. Customer and Address Data

Stores customer profiles and one or more delivery addresses per customer, including street details, administrative divisions, and geographic coordinates for navigation (`customer`, `customer_address`).

0.11.3. Delivery Order and Package Data

Stores customer delivery orders, requested delivery times, estimated arrivals, completion timestamps, and detailed package dimensions and weights (`delivery_order`, `package`, `order_status_history`).

0.11.4. Drone and Station Data

Stores drone physical specifications, operational statuses, battery levels, landing station coordinates, station capacity, status transition history, and operator assignments (`drone`, `landing_station`, `station_status_history`, `station_operator_assignment`).

0.11.5. Delivery Activity and Event Data

Stores discrete execution attempts, drone assignments, departure/destination stations, previous activity links for rescheduling, and station package handling events (`delivery_activity`, `package_station_event`).

0.11.6. Tracking and Exception Data

Stores high-frequency flight telemetry (coordinates, altitude, speed, battery) and records flight incidents, failure categories, and resolutions (`tracking_record`, `delivery_exception`).

0.11.7. Confirmation Data

Stores formal proof of delivery, recipient name, contact number, confirmation method, and object storage key for digital signatures or photos (`delivery_confirmation`).

0.11.8. Route and AI Recommendation Data

Stores planned waypoint paths and AI recommendation outputs including drone selection, duration predictions, and confidence scores (`delivery_route`, `route_point`, `ai_recommendation`).

0.11.9. Audit Data

Preserves an immutable audit trail capturing actor IDs, action types, table names, affected record IDs, JSON before/after snapshots, and IP addresses (`audit_log`).

0.12. Requirement-to-Entity Traceability

TABLE 1 maps each identified requirement and business rule to the database entity or relation that satisfies it. This matrix verifies that the database design completely covers all specifications without omissions.

0.13. Chapter Summary

This chapter established the comprehensive database requirements analysis for the SmartDroneDelivery platform. The five system actors and their specific data responsibilities were defined. Functional requirements across five core operational modules and non-functional constraints were specified.

The complete delivery lifecycle was articulated from order placement through package station pickup, drone flight tracking, exception handling, rescheduling, and final confirmation. Thirty-nine formal business rules (BR-01 through BR-39) were defined to eliminate ambiguity. Finally, the traceability matrix demonstrated how all requirements map directly to twenty-three relational entities, establishing a robust foundation for the conceptual, logical, and physical database design in Chapter 0.13.

Table 1. Requirement-to-entity traceability matrix

Requirement / Business Rule	Entity / Table	Status
Customer multiple delivery addresses (BR-01, 05)	customer_address	Satisfied
Customer order placement (BR-02, 03, 04)	delivery_order	Satisfied
Package physical specifications (BR-06, 07, 08, 09)	package	Satisfied
Order status transition tracking (BR-10, 11, 12, 13)	order_status_history	Satisfied
Drone operational capacity (BR-15, 17, 18)	drone	Satisfied
Landing station capacity and status (BR-25, 26)	landing_station, station_status_history	Satisfied
Station operator staffing (BR-27, 28)	station_operator_assignment	Satisfied
Delivery execution attempt (BR-16, 19, 20, 23, 24)	delivery_activity	Satisfied
Rescheduling attempt linkage (BR-21, 22)	delivery_activity.previous_activity	Satisfied
Station package handling / pickup (BR-29, 30, 31, 32)	package_station_event	Satisfied
In-flight telemetry tracking (BR-12)	tracking_record	Satisfied
Delivery failure / incident capture (BR-13, 21)	delivery_exception	Satisfied
At-most-one confirmation with proof (BR-33, 34, 35, 36)	delivery_confirmation	Satisfied
Role-based access control	system_user, role, permission, bridge tables	Satisfied
Audit logging and traceability (BR-37, 38, 39)	audit_log	Satisfied
Planned route and waypoints	delivery_route, route_point	Satisfied
AI-assisted recommendations	ai_recommendation	Satisfied

Database Design

0.14. Conceptual Database Design

This section presents the conceptual database model for the SmartDroneDelivery platform. The conceptual model identifies the main entities, their attributes, the relationships between entities, and the cardinalities of those relationships. The conceptual model serves as the basis for the logical relational schema developed in Section 0.15.

0.14.1. Entity Identification

Based on the requirements and business rules documented in Chapter 0.4, the following twenty-three entities are identified. They are organized into seven functional groups to ensure comprehensive coverage without operational redundancy.

1. User and Authorization Group.

- **system_user** — Stores authentication credentials and account statuses for all actors. Avoids reserved SQL keywords.
- **role** — Defines standard operational roles (CUSTOMER, DISPATCHER, STATION_OPERATOR, LOGISTICS_MANAGER, SYSTEM_ADMIN).
- **permission** — Defines granular permissions (e.g., CREATE_ORDER, ASSIGN_DRONE, CONFIRM_PICKUP).
- **user_role** — Bridge table implementing the many-to-many relationship between users and roles.
- **role_permission** — Bridge table implementing the many-to-many relationship between roles and permissions.

2. Customer and Address Group.

- **customer** — Stores customer profile information, optionally linked to a system user account.
- **customer_address** — Stores one or more delivery addresses per customer, including precise geographic coordinates for drone navigation.

3. *Delivery Order and Package Group.*

- **delivery_order** — Represents customer delivery requests, tracking customer requested time, estimated arrival, and lifecycle state.
- **package** — Stores physical package specifications (weight and dimensions in centimeters) associated with an order.
- **order_status_history** — Append-only record capturing every status transition of a delivery order.

4. *Landing Station and Drone Management Group.*

- **landing_station** — Stores physical takeoff/landing stations, coordinates, and drone capacity.
- **station_status_history** — Records the history of station status transitions (maintenance, availability).
- **station_operator_assignment** — Records staffing assignments of station operators to landing stations over defined time periods.
- **drone** — Stores drone serial numbers, models, operational statuses, battery levels, and payload capacities.

5. *Delivery Execution and Telemetry Group.*

- **delivery_activity** — Central operational entity representing each discrete execution attempt for an order, linking the drone, departure station, and destination station.
- **package_station_event** — Records package physical handling events at landing stations (arrival, pickup confirmation, loading, unloading).
- **tracking_record** — Stores high-frequency GPS and telemetry data captured during drone flight.
- **delivery_exception** — Records flight failures, anomalies, or disruptions during a delivery activity.

6. *Delivery Confirmation Group.*

- **delivery_confirmation** — Records formal handover proof upon successful delivery, linked 1:0..1 with the order and referencing digital proof in object storage.

7. Audit, Route, and AI Assistance Group.

- **audit_log** — Immutable system audit trail capturing user actions, before/after JSON snapshots, and IP addresses.
- **delivery_route** — Stores planned flight path metadata for a delivery activity.
- **route_point** — Stores ordered waypoints along a planned delivery route.
- **ai_recommendation** — Stores persistent outputs of AI recommendation models (drone recommendations and ETA predictions).

0.14.2. Entity Attributes

This section specifies the attributes and data types of each identified entity.

system_user. Stores user account credentials and authentication state.

- **user_id** — Primary key (BIGINT GENERATED ALWAYS AS IDENTITY).
- **username** — Unique username (VARCHAR(100)).
- **password_hash** — Cryptographic password hash (VARCHAR(255)).
- **email** — Unique email address (VARCHAR(255)).
- **phone_number** — Unique phone number (VARCHAR(20)).
- **is_active** — Account active flag (BOOLEAN).
- **created_at, updated_at** — Account audit timestamps (TIMESTAMPTZ).

role. Defines system roles.

- **role_id** — Primary key (BIGINT).
- **role_name** — Unique role identifier (VARCHAR(50)).
- **description** — Description of role responsibilities (VARCHAR(255)).

permission. Defines functional permissions.

- **permission_id** — Primary key (BIGINT).
- **permission_name** — Unique permission code (VARCHAR(100)).
- **description** — Permission explanation (VARCHAR(255)).

user_role. Associates users with roles.

- *user_id, role_id* — Composite primary key and foreign keys.
- *assigned_at* — Timestamp of role assignment (TIMESTAMPTZ).

role_permission. Associates roles with permissions.

- *role_id, permission_id* — Composite primary key and foreign keys.

customer. Stores customer profiles.

- *customer_id* — Primary key (BIGINT).
- *user_id* — Foreign key to *system_user* (BIGINT UNIQUE NULL).
- *customer_name* — Full name (VARCHAR(100)).
- *phone_number* — Primary contact number (VARCHAR(20) UNIQUE).
- *email* — Contact email (VARCHAR(255) UNIQUE).
- *created_at* — Registration timestamp (TIMESTAMPTZ).

customer_address. Stores customer delivery locations.

- *address_id* — Primary key (BIGINT).
- *customer_id* — Foreign key to *customer* (BIGINT NOT NULL).
- *address_label* — Label such as “Home” or “Office” (VARCHAR(50)).
- *address_line* — Street address (VARCHAR(255) NOT NULL).
- *ward, district, city* — Administrative divisions (VARCHAR(100)).
- *latitude, longitude* — Coordinates (NUMERIC(9,6)).
- *is_default* — Default address flag (BOOLEAN NOT NULL DEFAULT FALSE).
- *created_at* — Creation timestamp (TIMESTAMPTZ).

delivery_order. Represents delivery requests.

- *order_id* — Primary key (BIGINT).
- *customer_id* — Foreign key to *customer* (BIGINT NOT NULL).
- *delivery_address_id* — Foreign key to *customer_address* (BIGINT NOT NULL).
- *order_status* — Current lifecycle state (VARCHAR(30) NOT NULL).
- *requested_delivery_time* — Customer desired delivery time (TIMESTAMPTZ).

- `estimated_arrival_at` — System estimated arrival time (TIMESTAMPTZ).
- `created_at` — Order placement timestamp (TIMESTAMPTZ NOT NULL).
- `completed_at`, `cancelled_at` — Finalization timestamps (TIMESTAMPTZ).
- `cancellation_reason` — Explanation if cancelled (VARCHAR(255)).

package. Stores package specifications.

- `package_id` — Primary key (BIGINT).
- `order_id` — Foreign key to `delivery_order` (BIGINT NOT NULL).
- `description` — Content description (VARCHAR(500)).
- `weight_kg` — Weight in kilograms (NUMERIC(10,2) > 0).
- `length_cm`, `width_cm`, `height_cm` — Dimensions in centimeters (NUMERIC(10,2) > 0).
- `package_status` — Status code (VARCHAR(30) NOT NULL).
- `created_at` — Creation timestamp (TIMESTAMPTZ).

order_status_history. Maintains immutable order lifecycle audit.

- `status_history_id` — Primary key (BIGINT).
- `order_id` — Foreign key to `delivery_order` (BIGINT NOT NULL).
- `status` — Recorded status value (VARCHAR(30) NOT NULL).
- `changed_by` — Foreign key to `system_user` (BIGINT).
- `reason` — Transition reason (VARCHAR(255)).
- `changed_at` — Transition timestamp (TIMESTAMPTZ NOT NULL).

landing_station. Stores physical landing stations.

- `station_id` — Primary key (BIGINT).
- `station_code` — Unique station code (VARCHAR(50) UNIQUE NOT NULL).
- `station_name` — Human-readable name (VARCHAR(100) NOT NULL).
- `address` — Physical address (VARCHAR(255)).
- `latitude`, `longitude` — Coordinates (NUMERIC(9,6)).
- `capacity` — Maximum drone capacity (INTEGER >= 0).
- `status` — Operational state (VARCHAR(30) NOT NULL).

- `created_at` — Creation timestamp (TIMESTAMPTZ).

station_status_history. Maintains station operational history.

- `history_id` — Primary key (BIGINT).
- `station_id` — Foreign key to `landing_station` (BIGINT NOT NULL).
- `status` — Recorded station state (VARCHAR(30) NOT NULL).
- `changed_by` — Foreign key to `system_user` (BIGINT).
- `reason` — Transition reason (VARCHAR(255)).
- `changed_at` — Transition timestamp (TIMESTAMPTZ NOT NULL).

station_operator_assignment. Records operator staffing at stations.

- `assignment_id` — Primary key (BIGINT).
- `station_id` — Foreign key to `landing_station` (BIGINT NOT NULL).
- `user_id` — Foreign key to `system_user` representing the operator (BIGINT NOT NULL).
- `assigned_from`, `assigned_to` — Assignment validity window (TIMESTAMPTZ).
- `is_active` — Flag indicating active assignment (BOOLEAN NOT NULL DEFAULT TRUE).
- `created_at` — Assignment timestamp (TIMESTAMPTZ).

drone. Stores drone specifications and status.

- `drone_id` — Primary key (BIGINT).
- `serial_number` — Physical identifier (VARCHAR(100) UNIQUE NOT NULL).
- `model` — Model designation (VARCHAR(100)).
- `status` — Operational status (VARCHAR(30) NOT NULL).
- `max_payload_kg` — Maximum payload capacity (NUMERIC(10,2) > 0).
- `battery_level` — Current battery level (NUMERIC(5,2) between 0 and 100).
- `station_id` — Foreign key to current `landing_station` (BIGINT NULL).
- `created_at` — Registration timestamp (TIMESTAMPTZ).

Note: Drones do not store an `order_id`; drone assignment is mediated via `delivery_activity`.

delivery_activity. The operational mediator representing delivery execution attempts.

- *activity_id* — Primary key (BIGINT).
- *order_id* — Foreign key to *delivery_order* (BIGINT NOT NULL).
- *drone_id* — Foreign key to *drone* (BIGINT NOT NULL).
- *pickup_station_id* — Foreign key to *origin landing_station* (BIGINT).
- *destination_station_id* — Foreign key to *destination landing_station* (BIGINT).
- *assigned_by* — Foreign key to *dispatching system_user* (BIGINT).
- *scheduled_at* — Planned execution time (TIMESTAMPTZ).
- *activity_status* — Execution state (VARCHAR(30) NOT NULL).
- *started_at*, *ended_at* — Execution timestamps (TIMESTAMPTZ).
- *previous_activity_id* — Self-referencing foreign key linking preceding failed attempt (BIGINT NULL).
- *created_at* — Creation timestamp (TIMESTAMPTZ).

package_station_event. Records package handling at stations.

- *event_id* — Primary key (BIGINT).
- *package_id* — Foreign key to *package* (BIGINT NOT NULL).
- *station_id* — Foreign key to *landing_station* (BIGINT NOT NULL).
- *event_type* — Handling event type (VARCHAR(30) NOT NULL).
- *performed_by* — Foreign key to station operator *system_user* (BIGINT).
- *occurred_at* — Event timestamp (TIMESTAMPTZ NOT NULL).
- *note* — Operational remarks (VARCHAR(255)).

tracking_record. Stores high-frequency in-flight telemetry.

- *tracking_id* — Primary key (BIGINT).
- *activity_id* — Foreign key to *delivery_activity* (BIGINT NOT NULL).
- *latitude*, *longitude* — Coordinates (NUMERIC(9,6)).
- *altitude_m* — Altitude in meters (NUMERIC(10,2) >= 0).
- *speed_kmh* — Ground speed in km/h (NUMERIC(10,2) >= 0).
- *battery_level* — Battery percentage (NUMERIC(5,2)).

- `recorded_at` — Telemetry capture timestamp (TIMESTAMPTZ NOT NULL).

delivery_confirmation. Formal delivery proof.

- `confirmation_id` — Primary key (BIGINT).
- `order_id` — Foreign key to `delivery_order` (BIGINT UNIQUE NOT NULL).
- `activity_id` — Foreign key to `successful_delivery_activity` (BIGINT UNIQUE NOT NULL).
- `confirmed_by` — Foreign key to `verifying_system_user` (BIGINT).
- `receiver_name` — Name of package recipient (VARCHAR(100) NOT NULL).
- `receiver_phone` — Recipient contact phone (VARCHAR(20)).
- `confirmation_method` — Method code (VARCHAR(30) NOT NULL).
- `evidence_object_key` — Object storage key for digital signature/photo proof (VARCHAR(255)).
- `confirmed_at` — Handover timestamp (TIMESTAMPTZ NOT NULL).
- `note` — Delivery remarks (VARCHAR(500)).

delivery_exception. Records flight failures and incidents.

- `exception_id` — Primary key (BIGINT).
- `activity_id` — Foreign key to `delivery_activity` (BIGINT NOT NULL).
- `exception_type` — Incident category (VARCHAR(50) NOT NULL).
- `description` — Detailed incident narrative (TEXT).
- `detected_at` — Incident detection timestamp (TIMESTAMPTZ NOT NULL).
- `resolved_at` — Resolution timestamp (TIMESTAMPTZ).
- `resolved_by` — Foreign key to `resolving_system_user` (BIGINT).
- `resolution` — Corrective action explanation (TEXT).

audit_log. System-wide security and modification audit log.

- `log_id` — Primary key (BIGINT).
- `user_id` — Foreign key to `actor_system_user` (BIGINT NULL).
- `entity_name` — Target table name (VARCHAR(100) NOT NULL).
- `record_id` — Target record primary key (BIGINT).
- `action_type` — Operation type (VARCHAR(50) NOT NULL).

- `old_value`, `new_value` — Before/after JSON snapshots (JSONB).
- `action_time` — Audit timestamp (TIMESTAMPTZ NOT NULL).
- `ip_address` — Client network address (INET).

delivery_route and route_point. Planned flight trajectories and waypoints.

- `delivery_route`: `route_id` (PK), `activity_id` (FK UNIQUE), `total_distance_km`, `estimated_duration_min`, `created_at`.
- `route_point`: `route_point_id` (PK), `route_id` (FK), `sequence_number`, `latitude`, `longitude`, `altitude_m`.

ai_recommendation. Persistent outputs from AI analytics services.

- `recommendation_id` — Primary key (BIGINT).
- `order_id` — Foreign key to `delivery_order` (BIGINT NOT NULL).
- `activity_id` — Foreign key to `delivery_activity` (BIGINT NULL).
- `recommendation_type` — Recommendation category (VARCHAR(50) NOT NULL).
- `recommended_drone_id` — Foreign key to suggested drone (BIGINT NULL).
- `estimated_duration_min` — Model estimated flight minutes (NUMERIC(10,2)).
- `confidence_score` — Confidence level (NUMERIC(5,4) between 0 and 1).
- `model_version` — Version identifier of the AI service (VARCHAR(50)).
- `created_at` — Recommendation generation timestamp (TIMESTAMPTZ).

0.14.3. Relationships and Cardinalities

The relationships and cardinalities between the twenty-three entities are defined as follows, strictly conforming to the business rules:

- **customer 1 : N customer_address** — One customer owns many addresses; each address belongs to exactly one customer (BR-01).
- **customer 1 : N delivery_order** — One customer creates many delivery orders; each order belongs to one customer (BR-02).
- **customer_address 1 : N delivery_order** — One address can be the destination for many orders; each order specifies one destination address (BR-03, BR-04).
- **delivery_order 1 : N package** — One order contains one or more packages; each package belongs to one order (BR-06, BR-07).

- **delivery_order** 1 : N **order_status_history** — One order accumulates multiple historical status records (BR-10, BR-11).
- **delivery_order** 1 : N **delivery_activity** — One order can have multiple execution attempts (initial and rescheduled) (BR-19, BR-20).
- **delivery_order** 1 : 0..1 **delivery_confirmation** — One order has at most one successful delivery confirmation (BR-33).
- **delivery_activity** 1 : 0..1 **delivery_confirmation** — A confirmation is tied to exactly one successful delivery activity attempt (BR-35).
- **delivery_activity** 0..1 : 1 **delivery_activity** — Rescheduled activities link to their preceding attempt via `previous_activity_id` (BR-22).
- **drone** 1 : N **delivery_activity** — A drone serves many delivery activities over time, but only one active flight at any instant (BR-15, BR-16).
- **landing_station** 1 : N **drone** — A station accommodates multiple parked drones (BR-25).
- **landing_station** 1 : N **station_status_history** — A station preserves its operational status changes (BR-26).
- **landing_station** 1 : N **station_operator_assignment** — A station has multiple operator staffing assignments over time (BR-27).
- **system_user** 1 : N **station_operator_assignment** — A station operator user can be assigned to multiple stations over time (BR-27).
- **package** 1 : N **package_station_event** — A package undergoes multiple station handling events (arrival, pickup, loading) (BR-29, BR-30).
- **landing_station** 1 : N **package_station_event** — A station processes events for many packages (BR-31).
- **delivery_activity** 1 : N **tracking_record** — Each activity generates an immutable series of flight telemetry points (BR-12).
- **delivery_activity** 1 : N **delivery_exception** — An activity may encounter zero, one, or multiple exception incidents (BR-13, BR-21).
- **delivery_activity** 1 : 0..1 **delivery_route** — An activity has at most one planned route.
- **delivery_route** 1 : N **route_point** — A route is composed of an ordered sequence of waypoints.
- **delivery_order** 1 : N **ai_recommendation** — An order may receive recommendations from multiple AI models over time.
- **system_user** N : N **role** — Mediated by **user_role**.

- **role** N : N **permission** — Mediated by **role_permission**.
- **system_user** 1 : N **audit_log** — Users generate audit log entries across the system (BR-37, BR-38).

0.15. Logical Database Design

This section derives the logical relational schema from the conceptual model. The relational schema is presented in standard notation, with primary keys denoted by (PK), foreign keys by (FK), and unique constraints by UNIQUE.

0.15.1. Relational Schema

SYSTEM_USER(user_id PK, username UNIQUE, password_hash,
email UNIQUE, phone_number UNIQUE, is_active,
created_at, updated_at)

ROLE(role_id PK, role_name UNIQUE, description)

PERMISSION(permission_id PK, permission_name UNIQUE,
description)

USER_ROLE(user_id PK/FK, role_id PK/FK, assigned_at)

ROLE_PERMISSION(role_id PK/FK, permission_id PK/FK)

CUSTOMER(customer_id PK, user_id FK UNIQUE,
customer_name, phone_number UNIQUE, email UNIQUE,
created_at)

CUSTOMER_ADDRESS(address_id PK, customer_id FK,
address_label, address_line, ward, district, city,
latitude, longitude, is_default, created_at)

DELIVERY_ORDER(order_id PK, customer_id FK,
delivery_address_id FK, order_status,
requested_delivery_time, estimated_arrival_at,
created_at, completed_at, cancelled_at,
cancellation_reason)

PACKAGE(package_id PK, order_id FK, description,
weight_kg, length_cm, width_cm, height_cm,
package_status, created_at)

ORDER_STATUS_HISTORY(status_history_id PK, order_id FK,
status, changed_by FK, reason, changed_at)

LANDING_STATION(station_id PK, station_code UNIQUE,
station_name, address, latitude, longitude,
capacity, status, created_at)

STATION_STATUS_HISTORY(history_id PK, station_id FK,
status, changed_by FK, reason, changed_at)

STATION_OPERATOR_ASSIGNMENT(assignment_id PK,
station_id FK, user_id FK, assigned_from,
assigned_to, is_active, created_at)

DRONE(drone_id PK, serial_number UNIQUE, model, status,
max_payload_kg, battery_level, station_id FK,
created_at)

DELIVERY_ACTIVITY(activity_id PK, order_id FK,
drone_id FK, pickup_station_id FK,
destination_station_id FK, assigned_by FK,
scheduled_at, activity_status, started_at, ended_at,
previous_activity_id FK, created_at)

PACKAGE_STATION_EVENT(event_id PK, package_id FK,
station_id FK, event_type, performed_by FK,
occurred_at, note)

TRACKING_RECORD(tracking_id PK, activity_id FK,
latitude, longitude, altitude_m, speed_kmh,
battery_level, recorded_at)

DELIVERY_CONFIRMATION(confirmation_id PK,
order_id FK UNIQUE, activity_id FK UNIQUE,

confirmed_by FK, receiver_name, receiver_phone,
confirmation_method, evidence_object_key,
confirmed_at, note)

DELIVERY_EXCEPTION(exception_id PK, activity_id FK,
exception_type, description, detected_at,
resolved_at, resolved_by FK, resolution)

AUDIT_LOG(log_id PK, user_id FK, entity_name,
record_id, action_type, old_value, new_value,
action_time, ip_address)

DELIVERY_ROUTE(route_id PK, activity_id FK UNIQUE,
total_distance_km, estimated_duration_min,
created_at)

ROUTE_POINT(route_point_id PK, route_id FK,
sequence_number, latitude, longitude, altitude_m)

AI_RECOMMENDATION(recommendation_id PK, order_id FK,
activity_id FK, recommendation_type,
recommended_drone_id FK, estimated_duration_min,
confidence_score, model_version, created_at)

0.15.2. Primary Keys, Foreign Keys, and Candidate Keys

Every base table possesses a single-column surrogate primary key (BIGINT GENERATED ALWAYS AS IDENTITY). Bridge tables (user_role, role_permission) employ composite primary keys. Candidate natural keys are protected with UNIQUE constraints:

- system_user.username, system_user.email, system_user.phone_number
- customer.phone_number, customer.email
- drone.serial_number
- landing_station.station_code
- delivery_confirmation.order_id and delivery_confirmation.activity_id
- delivery_route.activity_id
- route_point(route_id, sequence_number)

0.15.3. Functional Dependencies and Normalization

All twenty-three relations satisfy Third Normal Form (3NF):

First Normal Form (1NF)... All column values are atomic. Address components are separated into discrete attributes; dimensions are stored as individual numeric values (`length_cm`, `width_cm`, `height_cm`); and multiple addresses or packages are represented as separate relational rows rather than composite strings.

Second Normal Form (2NF)... Every non-key attribute is fully functionally dependent on the entire primary key. In composite-key tables (`user_role`, `role_permission`), non-key attributes depend on both component keys.

Third Normal Form (3NF)... No transitive dependencies exist between non-key attributes. For example, `delivery_order` references `customer_id` and does not store customer contact details; `delivery_confirmation` does not store customer or drone data; and `tracking_record` references `activity_id` without redundantly storing drone attributes.

0.15.4. Integrity Constraints

The relational schema enforces six categories of integrity constraints:

1. **Entity Integrity:** Guaranteed by surrogate primary keys that are automatically generated, non-null, and unique across all tables.
2. **Referential Integrity:** Enforced via foreign key constraints. Operational master data (`customer`, `drone`, `landing_station`) use `ON DELETE RESTRICT` to prevent accidental deletion of operational history. Security bridge tables use `ON DELETE CASCADE`. User audit references use `ON DELETE SET NULL` to preserve audit trails if user accounts are archived.
3. **Domain Integrity:** Column data types and explicit `CHECK` constraints ensure valid domain values (coordinates within $[-90, 90]$ and $[-180, 180]$, positive dimensions and payload capacities, battery percentages between 0 and 100).
4. **Uniqueness Constraints:** Natural identifiers and 1:0..1 cardinalities are strictly enforced by database-level `UNIQUE` indexes.
5. **Business Integrity:** Total package weight must not exceed assigned drone capacity (BR-18); address must belong to the ordering customer (BR-04).

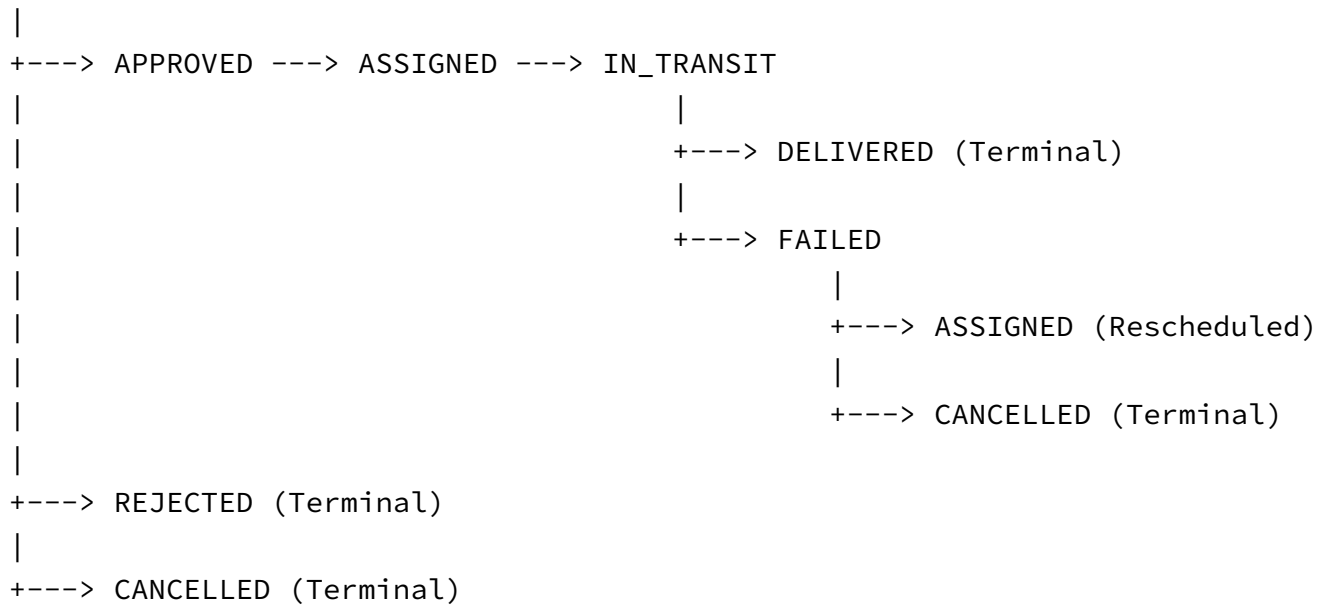
6. **Temporal Integrity:** In delivery_activity, started_at < ended_at; in station_operator_assigned, assigned_from < assigned_to.

0.15.5. State Transition Lifecycle Models

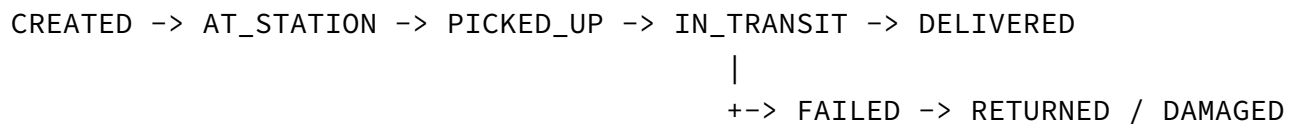
The system defines formal lifecycles for its three central stateful entities:

Delivery Order Lifecycle.

PENDING



Package Lifecycle.



Delivery Activity Lifecycle.



0.16. Physical Database Design

This section details the physical PostgreSQL implementation. All tables employ BIGINT GENERATED ALWAYS AS IDENTITY for primary keys, TIMESTAMPTZ for time fields, and appropriate numeric and geometric domains.

0.16.1. CREATE TABLE Statements

-- 1. User and Authorization Group

```
CREATE TABLE system_user (  
    user_id          BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,  
    username         VARCHAR(100) NOT NULL UNIQUE,  
    password_hash    VARCHAR(255) NOT NULL,  
    email            VARCHAR(255) UNIQUE,  
    phone_number     VARCHAR(20)  UNIQUE,  
    is_active        BOOLEAN      NOT NULL DEFAULT TRUE,  
    created_at       TIMESTAMPTZ  NOT NULL DEFAULT CURRENT_TIMESTAMP,  
    updated_at       TIMESTAMPTZ  NOT NULL DEFAULT CURRENT_TIMESTAMP  
);
```

```
CREATE TABLE role (  
    role_id          BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,  
    role_name        VARCHAR(50)  NOT NULL UNIQUE,  
    description       VARCHAR(255)  
);
```

```
CREATE TABLE permission (  
    permission_id    BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,  
    permission_name  VARCHAR(100) NOT NULL UNIQUE,  
    description       VARCHAR(255)  
);
```

```
CREATE TABLE user_role (  
    user_id          BIGINT      NOT NULL,  
    role_id          BIGINT      NOT NULL,  
    assigned_at      TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP,  
    PRIMARY KEY (user_id, role_id),  
    CONSTRAINT fk_ur_user FOREIGN KEY (user_id)  
        REFERENCES system_user(user_id) ON DELETE CASCADE,  
    CONSTRAINT fk_ur_role FOREIGN KEY (role_id)  
        REFERENCES role(role_id) ON DELETE CASCADE  
);
```

```
CREATE TABLE role_permission (  

```

```

    role_id          BIGINT NOT NULL,
    permission_id BIGINT NOT NULL,
    PRIMARY KEY (role_id, permission_id),
    CONSTRAINT fk_rp_role FOREIGN KEY (role_id)
        REFERENCES role(role_id) ON DELETE CASCADE,
    CONSTRAINT fk_rp_perm FOREIGN KEY (permission_id)
        REFERENCES permission(permission_id) ON DELETE CASCADE
);

```

-- 2. Customer and Address Group

```

CREATE TABLE customer (
    customer_id  BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    user_id      BIGINT          UNIQUE,
    customer_name VARCHAR(100) NOT NULL,
    phone_number VARCHAR(20)  UNIQUE,
    email        VARCHAR(255) UNIQUE,
    created_at   TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT fk_customer_user FOREIGN KEY (user_id)
        REFERENCES system_user(user_id) ON DELETE RESTRICT
);

```

```

CREATE TABLE customer_address (
    address_id  BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    customer_id BIGINT          NOT NULL,
    address_label VARCHAR(50),
    address_line VARCHAR(255) NOT NULL,
    ward        VARCHAR(100),
    district    VARCHAR(100),
    city        VARCHAR(100),
    latitude    NUMERIC(9,6) CHECK (latitude BETWEEN -90 AND 90),
    longitude    NUMERIC(9,6) CHECK (longitude BETWEEN -180 AND 180),
    is_default   BOOLEAN          NOT NULL DEFAULT FALSE,
    created_at   TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT fk_addr_customer FOREIGN KEY (customer_id)
        REFERENCES customer(customer_id) ON DELETE RESTRICT
);

```

-- 3. Landing Station and Operator Group

```
CREATE TABLE landing_station (  
    station_id    BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,  
    station_code  VARCHAR(50)  NOT NULL UNIQUE,  
    station_name  VARCHAR(100) NOT NULL,  
    address       VARCHAR(255),  
    latitude      NUMERIC(9,6) CHECK (latitude BETWEEN -90 AND 90),  
    longitude     NUMERIC(9,6) CHECK (longitude BETWEEN -180 AND 180),  
    capacity      INTEGER      NOT NULL DEFAULT 0 CHECK (capacity >= 0),  
    status        VARCHAR(30)  NOT NULL  
        CHECK (status IN ('AVAILABLE', 'FULL', 'MAINTENANCE', 'OFFLINE')),  
    created_at    TIMESTAMPTZ  NOT NULL DEFAULT CURRENT_TIMESTAMP  
);
```

```
CREATE TABLE station_status_history (  
    history_id    BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,  
    station_id    BIGINT      NOT NULL,  
    status        VARCHAR(30) NOT NULL,  
    changed_by    BIGINT,  
    reason        VARCHAR(255),  
    changed_at    TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP,  
    CONSTRAINT fk_ssh_station FOREIGN KEY (station_id)  
        REFERENCES landing_station(station_id) ON DELETE RESTRICT,  
    CONSTRAINT fk_ssh_user FOREIGN KEY (changed_by)  
        REFERENCES system_user(user_id) ON DELETE SET NULL  
);
```

```
CREATE TABLE station_operator_assignment (  
    assignment_id BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,  
    station_id    BIGINT      NOT NULL,  
    user_id       BIGINT      NOT NULL,  
    assigned_from TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP,  
    assigned_to   TIMESTAMPTZ,  
    is_active     BOOLEAN     NOT NULL DEFAULT TRUE,  
    created_at    TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP,  
    CONSTRAINT fk_soa_station FOREIGN KEY (station_id)  
        REFERENCES landing_station(station_id) ON DELETE RESTRICT,
```

```

CONSTRAINT fk_soa_user FOREIGN KEY (user_id)
REFERENCES system_user(user_id) ON DELETE RESTRICT,
CONSTRAINT chk_soa_interval CHECK (assigned_to IS NULL OR assigned_from < assigned_to);
);

```

-- 4. Drone Group

```

CREATE TABLE drone (
    drone_id          BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    serial_number     VARCHAR(100) NOT NULL UNIQUE,
    model             VARCHAR(100),
    status            VARCHAR(30) NOT NULL
        CHECK (status IN ('AVAILABLE', 'BUSY', 'MAINTENANCE', 'OFFLINE')),
    max_payload_kg    NUMERIC(10,2) NOT NULL CHECK (max_payload_kg > 0),
    battery_level     NUMERIC(5,2) CHECK (battery_level BETWEEN 0 AND 100),
    station_id        BIGINT,
    created_at        TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT fk_drone_station FOREIGN KEY (station_id)
        REFERENCES landing_station(station_id) ON DELETE SET NULL
);

```

-- 5. Delivery Order and Package Group

```

CREATE TABLE delivery_order (
    order_id          BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    customer_id       BIGINT NOT NULL,
    delivery_address_id BIGINT NOT NULL,
    order_status       VARCHAR(30) NOT NULL
        CHECK (order_status IN (
            'PENDING', 'APPROVED', 'REJECTED', 'ASSIGNED',
            'IN_TRANSIT', 'DELIVERED', 'FAILED', 'CANCELLED')),
    requested_delivery_time TIMESTAMPTZ,
    estimated_arrival_at   TIMESTAMPTZ,
    created_at          TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP,
    completed_at         TIMESTAMPTZ,
    cancelled_at         TIMESTAMPTZ,
    cancellation_reason   VARCHAR(255),
    CONSTRAINT fk_order_customer FOREIGN KEY (customer_id)

```

```

REFERENCES customer(customer_id) ON DELETE RESTRICT,
CONSTRAINT fk_order_address FOREIGN KEY (delivery_address_id)
REFERENCES customer_address(address_id) ON DELETE RESTRICT
);

```

```

CREATE TABLE package (
package_id      BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
order_id        BIGINT          NOT NULL,
description     VARCHAR(500),
weight_kg       NUMERIC(10,2) NOT NULL CHECK (weight_kg > 0),
length_cm       NUMERIC(10,2) CHECK (length_cm > 0),
width_cm        NUMERIC(10,2) CHECK (width_cm > 0),
height_cm       NUMERIC(10,2) CHECK (height_cm > 0),
package_status  VARCHAR(30)     NOT NULL
CHECK (package_status IN (
    'CREATED', 'AT_STATION', 'PICKED_UP', 'IN_TRANSIT',
    'DELIVERED', 'FAILED', 'RETURNED', 'DAMAGED')),
created_at      TIMESTAMPTZ     NOT NULL DEFAULT CURRENT_TIMESTAMP,
CONSTRAINT fk_pkg_order FOREIGN KEY (order_id)
REFERENCES delivery_order(order_id) ON DELETE RESTRICT
);

```

```

CREATE TABLE order_status_history (
status_history_id BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
order_id          BIGINT          NOT NULL,
status            VARCHAR(30) NOT NULL,
changed_by        BIGINT,
reason            VARCHAR(255),
changed_at        TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP,
CONSTRAINT fk_osh_order FOREIGN KEY (order_id)
REFERENCES delivery_order(order_id) ON DELETE RESTRICT,
CONSTRAINT fk_osh_user FOREIGN KEY (changed_by)
REFERENCES system_user(user_id) ON DELETE SET NULL
);

```

-- 6. Delivery Execution and Event Group

```

CREATE TABLE delivery_activity (

```

```

activity_id          BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
order_id             BIGINT          NOT NULL,
drone_id             BIGINT          NOT NULL,
pickup_station_id    BIGINT,
destination_station_id BIGINT,
assigned_by          BIGINT,
scheduled_at         TIMESTAMPTZ,
activity_status      VARCHAR(30) NOT NULL
    CHECK (activity_status IN (
        'ASSIGNED', 'PREPARING', 'IN_TRANSIT',
        'DELIVERED', 'FAILED', 'CANCELLED')),
started_at           TIMESTAMPTZ,
ended_at             TIMESTAMPTZ,
previous_activity_id BIGINT,
created_at           TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP,
CONSTRAINT fk_act_order FOREIGN KEY (order_id)
    REFERENCES delivery_order(order_id) ON DELETE RESTRICT,
CONSTRAINT fk_act_drone FOREIGN KEY (drone_id)
    REFERENCES drone(drone_id) ON DELETE RESTRICT,
CONSTRAINT fk_act_pickup FOREIGN KEY (pickup_station_id)
    REFERENCES landing_station(station_id) ON DELETE SET NULL,
CONSTRAINT fk_act_dest FOREIGN KEY (destination_station_id)
    REFERENCES landing_station(station_id) ON DELETE SET NULL,
CONSTRAINT fk_act_user FOREIGN KEY (assigned_by)
    REFERENCES system_user(user_id) ON DELETE SET NULL,
CONSTRAINT fk_act_prev FOREIGN KEY (previous_activity_id)
    REFERENCES delivery_activity(activity_id) ON DELETE SET NULL,
CONSTRAINT chk_act_time CHECK (ended_at IS NULL OR started_at <= ended_at)
);

```

```

CREATE TABLE package_station_event (
    event_id          BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    package_id        BIGINT          NOT NULL,
    station_id        BIGINT          NOT NULL,
    event_type        VARCHAR(30) NOT NULL
    CHECK (event_type IN ('ARRIVED', 'PICKUP_CONFIRMED', 'LOADED', 'UNLOADED'))
    performed_by      BIGINT,
    occurred_at       TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP,

```

```

note          VARCHAR(255),
CONSTRAINT fk_pse_package FOREIGN KEY (package_id)
  REFERENCES package(package_id) ON DELETE RESTRICT,
CONSTRAINT fk_pse_station FOREIGN KEY (station_id)
  REFERENCES landing_station(station_id) ON DELETE RESTRICT,
CONSTRAINT fk_pse_user FOREIGN KEY (performed_by)
  REFERENCES system_user(user_id) ON DELETE SET NULL
);

CREATE TABLE tracking_record (
  tracking_id    BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  activity_id    BIGINT          NOT NULL,
  latitude       NUMERIC(9,6) CHECK (latitude BETWEEN -90 AND 90),
  longitude      NUMERIC(9,6) CHECK (longitude BETWEEN -180 AND 180),
  altitude_m     NUMERIC(10,2) CHECK (altitude_m >= 0),
  speed_kmh      NUMERIC(10,2) CHECK (speed_kmh >= 0),
  battery_level  NUMERIC(5,2)  CHECK (battery_level BETWEEN 0 AND 100),
  recorded_at    TIMESTAMPTZ   NOT NULL,
  CONSTRAINT fk_tr_activity FOREIGN KEY (activity_id)
    REFERENCES delivery_activity(activity_id) ON DELETE RESTRICT
);

CREATE TABLE delivery_confirmation (
  confirmation_id BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  order_id        BIGINT          NOT NULL UNIQUE,
  activity_id     BIGINT          NOT NULL UNIQUE,
  confirmed_by    BIGINT,
  receiver_name   VARCHAR(100) NOT NULL,
  receiver_phone  VARCHAR(20),
  confirmation_method VARCHAR(30) NOT NULL,
  evidence_object_key VARCHAR(255),
  confirmed_at    TIMESTAMPTZ   NOT NULL DEFAULT CURRENT_TIMESTAMP,
  note            VARCHAR(500),
  CONSTRAINT fk_conf_order FOREIGN KEY (order_id)
    REFERENCES delivery_order(order_id) ON DELETE RESTRICT,
  CONSTRAINT fk_conf_activity FOREIGN KEY (activity_id)
    REFERENCES delivery_activity(activity_id) ON DELETE RESTRICT,
  CONSTRAINT fk_conf_user FOREIGN KEY (confirmed_by)

```

```

REFERENCES system_user(user_id) ON DELETE SET NULL
);

CREATE TABLE delivery_exception (
    exception_id BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    activity_id BIGINT NOT NULL,
    exception_type VARCHAR(50) NOT NULL,
    description TEXT,
    detected_at TIMESTAMPTZ NOT NULL,
    resolved_at TIMESTAMPTZ,
    resolved_by BIGINT,
    resolution TEXT,
    CONSTRAINT fk_exc_activity FOREIGN KEY (activity_id)
        REFERENCES delivery_activity(activity_id) ON DELETE RESTRICT,
    CONSTRAINT fk_exc_resolver FOREIGN KEY (resolved_by)
        REFERENCES system_user(user_id) ON DELETE SET NULL
);

```

-- 7. Audit, Route, and AI Group

```

CREATE TABLE audit_log (
    log_id BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    user_id BIGINT,
    entity_name VARCHAR(100) NOT NULL,
    record_id BIGINT,
    action_type VARCHAR(50) NOT NULL,
    old_value JSONB,
    new_value JSONB,
    action_time TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP,
    ip_address INET,
    CONSTRAINT fk_al_user FOREIGN KEY (user_id)
        REFERENCES system_user(user_id) ON DELETE SET NULL
);

CREATE TABLE delivery_route (
    route_id BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    activity_id BIGINT NOT NULL UNIQUE,
    total_distance_km NUMERIC(12,3) CHECK (total_distance_km >= 0),

```

```

        estimated_duration_min NUMERIC(10,2) CHECK (estimated_duration_min >= 0),
        created_at             TIMESTAMPTZ   NOT NULL DEFAULT CURRENT_TIMESTAMP,
        CONSTRAINT fk_route_activity FOREIGN KEY (activity_id)
            REFERENCES delivery_activity(activity_id) ON DELETE RESTRICT
    );

```

```

CREATE TABLE route_point (
    route_point_id BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    route_id       BIGINT          NOT NULL,
    sequence_number INTEGER        NOT NULL,
    latitude       NUMERIC(9,6) CHECK (latitude BETWEEN -90 AND 90),
    longitude      NUMERIC(9,6) CHECK (longitude BETWEEN -180 AND 180),
    altitude_m     NUMERIC(10,2) CHECK (altitude_m >= 0),
    CONSTRAINT fk_rp_route FOREIGN KEY (route_id)
        REFERENCES delivery_route(route_id) ON DELETE CASCADE,
    CONSTRAINT uq_rp_seq UNIQUE (route_id, sequence_number)
);

```

```

CREATE TABLE ai_recommendation (
    recommendation_id BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    order_id          BIGINT          NOT NULL,
    activity_id       BIGINT,
    recommendation_type VARCHAR(50) NOT NULL,
    recommended_drone_id BIGINT,
    estimated_duration_min NUMERIC(10,2) CHECK (estimated_duration_min >= 0),
    confidence_score      NUMERIC(5,4) CHECK (confidence_score BETWEEN 0 AND 1),
    model_version         VARCHAR(50),
    created_at            TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT fk_ai_order FOREIGN KEY (order_id)
        REFERENCES delivery_order(order_id) ON DELETE RESTRICT,
    CONSTRAINT fk_ai_activity FOREIGN KEY (activity_id)
        REFERENCES delivery_activity(activity_id) ON DELETE SET NULL,
    CONSTRAINT fk_ai_drone FOREIGN KEY (recommended_drone_id)
        REFERENCES drone(drone_id) ON DELETE SET NULL
);

```

0.16.2. Index Strategy

Indexes are targeted at foreign keys and frequent query filters while avoiding redundant indexes on columns covered by unique constraints:

```
-- Foreign key indexes for operational joins
CREATE INDEX idx_order_customer      ON delivery_order(customer_id);
CREATE INDEX idx_order_address       ON delivery_order(delivery_address_id);
CREATE INDEX idx_osh_order           ON order_status_history(order_id);
CREATE INDEX idx_pkg_order           ON package(order_id);
CREATE INDEX idx_activity_order      ON delivery_activity(order_id);
CREATE INDEX idx_activity_drone      ON delivery_activity(drone_id);
CREATE INDEX idx_activity_prev       ON delivery_activity(previous_activity_id);
CREATE INDEX idx_tracking_activity   ON tracking_record(activity_id);
CREATE INDEX idx_exception_activity  ON delivery_exception(activity_id);
CREATE INDEX idx_ssh_station         ON station_status_history(station_id);
CREATE INDEX idx_soa_station         ON station_operator_assignment(station_id);
CREATE INDEX idx_soa_user            ON station_operator_assignment(user_id);
CREATE INDEX idx_pse_package         ON package_station_event(package_id);
CREATE INDEX idx_pse_station         ON package_station_event(station_id);
CREATE INDEX idx_audit_user          ON audit_log(user_id);
CREATE INDEX idx_audit_time          ON audit_log(action_time);

-- Composite index for recent flight telemetry queries
CREATE INDEX idx_tracking_activity_time
    ON tracking_record(activity_id, recorded_at DESC);
```

0.16.3. Representative Business Queries

The following queries confirm that the physical design directly answers operational needs:

Retrieve orders for a customer with latest status.

```
SELECT order_id, order_status, requested_delivery_time,
       estimated_arrival_at, created_at
FROM   delivery_order
WHERE  customer_id = :customer_id
ORDER BY created_at DESC;
```

Retrieve package pickup confirmation history at stations.

```

SELECT pse.event_id, pse.package_id, ls.station_name,
       pse.event_type, su.username AS operator_name, pse.occurred_at
FROM   package_station_event pse
JOIN   landing_station ls ON ls.station_id = pse.station_id
LEFT   JOIN system_user su ON su.user_id = pse.performed_by
WHERE  pse.package_id = :package_id
ORDER BY pse.occurred_at ASC;

```

Trace rescheduling chain for a delivery order.

```

SELECT activity_id, drone_id, activity_status,
       previous_activity_id, started_at, ended_at
FROM   delivery_activity
WHERE  order_id = :order_id
ORDER BY created_at ASC;

```

Retrieve flight telemetry for monitoring.

```

SELECT recorded_at, latitude, longitude, altitude_m, speed_kmh, battery_level
FROM   tracking_record
WHERE  activity_id = :activity_id
ORDER BY recorded_at DESC
LIMIT  50;

```

0.17. Chapter Summary

This chapter detailed the conceptual, logical, and physical database design for the SmartDroneDelivery platform. The model comprises twenty-three normalized relations organized into seven functional groups.

Key architectural enhancements include introducing `delivery_activity` to decouple orders from drones, adding `station_operator_assignment` and `package_station_event` to formalize station staffing and package pickup workflows, incorporating `previous_activity_id` to track rescheduling chains, and referencing external digital proof keys in `delivery_confirmation`.

The logical schema satisfies 3NF and enforces six categories of integrity constraints. Finally, complete PostgreSQL DDL statements and an index strategy were presented, providing a rigorous, efficient, and fully traceable implementation ready for comprehensive validation in Chapter 0.17.

Database Testing and Evaluation

0.18. Testing Objectives

The goal of database testing is to verify that the schema correctly enforces all integrity constraints defined in the design, that all thirty-nine business rules are reflected in the database behavior, and that the query patterns required by the application perform acceptably. Testing is organized into constraint tests, business rule tests, history tests, failed delivery rescheduling tests, station operation tests, query tests, and performance tests.

0.19. Constraint Tests

0.19.1. Primary Key Tests

Primary key constraints ensure that no two rows in a table share the same identifier.

Table 2. Primary key test cases

Test	Action	Expected	Result
PK-01	Insert a second customer row with a manually specified duplicate <code>customer_id</code> (identity column)	FAIL	PASS
PK-02	Insert two drone rows without specifying <code>drone_id</code> ; verify each receives a distinct identity value	PASS	PASS
PK-03	Insert a duplicate row into <code>user_role</code> with the same (<code>user_id</code> , <code>role_id</code>) composite key	FAIL	PASS
PK-04	Insert a duplicate row into <code>role_permission</code> with the same (<code>role_id</code> , <code>permission_id</code>)	FAIL	PASS

0.19.2. Foreign Key Tests

Foreign key constraints ensure that referencing rows cannot point to non-existent parent records and maintain referential integrity.

Table 3. Foreign key test cases

Test	Action	Expected	Result
FK-01	Insert a delivery_order with a customer_id that does not exist	FAIL	PASS
FK-02	Insert a delivery_activity with a drone_id that does not exist	FAIL	PASS
FK-03	Insert a tracking_record with an activity_id that does not exist	FAIL	PASS
FK-04	Insert a station_operator_assignment with an invalid station_id or user_id	FAIL	PASS
FK-05	Insert a package_station_event referencing a non-existent package_id	FAIL	PASS
FK-06	Insert a delivery_activity with a non-existent previous_activity_id	FAIL	PASS
FK-07	Attempt to delete a customer that has associated delivery_order rows	FAIL (RE-STRICT)	PASS
FK-08	Delete a role; verify corresponding user_role rows are removed by CASCADE	PASS	PASS

0.19.3. Unique Constraint Tests

Unique constraints enforce that specific column values or combinations are not duplicated across rows.

0.19.4. Check Constraint Tests

Check constraints enforce domain validity and temporal consistency on individual column values.

0.20. Business Rule Tests

0.20.1. Multiple Addresses per Customer (BR-01, BR-05)

Business Rule: A customer may have multiple delivery addresses, with at most one marked as default.

```
-- Setup customer
```

```
INSERT INTO customer (customer_name, phone_number, email)
```

Table 4. Unique constraint test cases

Test	Action	Expected	Result
UQ-01	Insert two drone rows with the same serial_number	FAIL	PASS
UQ-02	Insert two system_user rows with the same username	FAIL	PASS
UQ-03	Insert two landing_station rows with the same station_code	FAIL	PASS
UQ-04	Insert a second delivery_confirmation for the same order_id	FAIL	PASS
UQ-05	Insert a second delivery_confirmation for the same activity_id	FAIL	PASS
UQ-06	Insert two route_point rows for the same route_id with the same sequence_number	FAIL	PASS

```
VALUES ('Nguyen Van A', '0901000001', 'a@example.com');
```

```
-- Insert multiple addresses
```

```
INSERT INTO customer_address (customer_id, address_line, city, is_default)
VALUES (1, '10 Le Loi', 'Ho Chi Minh City', TRUE),
       (1, '25 Nguyen Hue', 'Ho Chi Minh City', FALSE),
       (1, '5 Tran Hung Dao', 'Hanoi', FALSE);
```

```
SELECT COUNT(*) FROM customer_address WHERE customer_id = 1;
```

```
-- Expected: 3
```

Expected: PASS. Three distinct address records are preserved and associated with customer 1.

0.20.2. Multiple Delivery Activities and Rescheduling (BR-19, BR-22)

Business Rule: When a delivery attempt fails, the failed activity record is preserved, and rescheduling creates a new delivery activity linked to the previous attempt via previous_activity_id.

```
-- First attempt fails
```

```
INSERT INTO delivery_activity
(order_id, drone_id, activity_status, started_at, ended_at, created_at)
VALUES (1, 1, 'FAILED',
       '2025-01-10 10:00:00+07', '2025-01-10 10:30:00+07',
       '2025-01-10 09:45:00+07');
```

Table 5. Check constraint test cases

Test	Action	Expected	Result
CK-01	Insert a package with weight_kg = -1	FAIL	PASS
CK-02	Insert a drone with battery_level = 105	FAIL	PASS
CK-03	Insert a customer_address with latitude = 95	FAIL	PASS
CK-04	Insert a landing_station with capacity = -5	FAIL	PASS
CK-05	Insert a delivery_order with order_status = 'UNKNOWN'	FAIL	PASS
CK-06	Insert a package with package_status = 'LOST' (not in allowed set)	FAIL	PASS
CK-07	Insert a package_station_event with invalid event_type	FAIL	PASS
CK-08	Insert a station_operator_assignment with assigned_to < assigned_from	FAIL	PASS
CK-09	Insert a delivery_activity with ended_at < started_at	FAIL	PASS

-- Record failure exception

INSERT INTO delivery_exception

(activity_id, exception_type, description, detected_at)
VALUES (1, 'WEATHER', 'Severe thunderstorm along flight path',
'2025-01-10 10:20:00+07');

-- Reschedule: create second activity linking to the first

INSERT INTO delivery_activity

(order_id, drone_id, activity_status, previous_activity_id, created_at)
VALUES (1, 2, 'ASSIGNED', 1, CURRENT_TIMESTAMP);

SELECT activity_id, activity_status, previous_activity_id

FROM delivery_activity

WHERE order_id = 1

ORDER BY created_at;

-- Expected:

-- Row 1: activity_id=1, status='FAILED', previous_activity_id=NULL

-- Row 2: activity_id=2, status='ASSIGNED', previous_activity_id=1

Expected: PASS. Both attempts are preserved in the database, establishing an unbroken operational audit trail for the order.

0.20.3. At Most One Confirmation per Order and Activity Link (BR-33, BR-35, BR-36)

Business Rule: A delivery order has at most one confirmation record, tied to the successful activity attempt and referencing digital proof in object storage.

```
-- Successful confirmation for activity 2
INSERT INTO delivery_confirmation
    (order_id, activity_id, receiver_name, receiver_phone,
     confirmation_method, evidence_object_key)
VALUES (1, 2, 'Tran Thi B', '0909000002', 'SIGNATURE',
        'proofs/2025/01/ord_1_sig.png');

-- Duplicate confirmation attempt for the same order must fail
INSERT INTO delivery_confirmation
    (order_id, activity_id, receiver_name, confirmation_method)
VALUES (1, 2, 'Tran Thi B', 'OTP');
-- Expected: FAIL due to UNIQUE(order_id)
```

Expected: PASS. The first insert succeeds; the second insert fails with a unique constraint violation on order_id.

0.20.4. Station Operator Staffing and Package Handling (BR-27, BR-28, BR-30, BR-31)

Business Rule: Station operators are assigned to stations, and package pickup events record the performing operator and station.

```
-- Assign operator to station 1
INSERT INTO station_operator_assignment (station_id, user_id, is_active)
VALUES (1, 3, TRUE);

-- Package arrival and pickup confirmation at station
INSERT INTO package_station_event
    (package_id, station_id, event_type, performed_by, note)
VALUES (1, 1, 'ARRIVED', 3, 'Package received at station dock'),
        (1, 1, 'PICKUP_CONFIRMED', 3, 'Barcodes verified'),
        (1, 1, 'LOADED', 3, 'Secured into drone payload bay');

SELECT event_type, occurred_at
FROM    package_station_event
WHERE   package_id = 1
ORDER  BY occurred_at ASC;
```

-- Expected: 3 events recorded in chronological order

Expected: PASS. All station handling events are recorded with operator and station references.

0.21. History Preservation Tests

0.21.1. Order Status History

Business Rule: Every order state transition creates an append-only row in order_status_history.

```
INSERT INTO order_status_history (order_id, status, changed_at)
VALUES
(1, 'PENDING',      '2025-01-10 08:00:00+07'),
(1, 'APPROVED',     '2025-01-10 09:00:00+07'),
(1, 'ASSIGNED',     '2025-01-10 09:30:00+07'),
(1, 'IN_TRANSIT',   '2025-01-10 10:00:00+07'),
(1, 'DELIVERED',    '2025-01-10 10:45:00+07');
```

```
SELECT COUNT(*) FROM order_status_history WHERE order_id = 1;
-- Expected: 5 rows
```

Expected: PASS. Five distinct chronological records are stored without overwriting.

0.21.2. Station Status History

Business Rule: Landing station availability and maintenance transitions must be preserved.

```
INSERT INTO station_status_history (station_id, status, reason, changed_at)
VALUES
(1, 'AVAILABLE',    NULL,          '2025-01-09 08:00:00+07'),
(1, 'MAINTENANCE',  'Pad resurfacing', '2025-01-09 14:00:00+07'),
(1, 'AVAILABLE',    'Work finished',   '2025-01-10 08:00:00+07');
```

```
SELECT COUNT(*) FROM station_status_history WHERE station_id = 1;
-- Expected: 3
```

Expected: PASS. All station state changes are preserved.

0.22. Audit Log Verification Test

Business Rule: Critical updates generate an immutable audit record containing old and new JSON states.

```
INSERT INTO audit_log
  (user_id, entity_name, record_id, action_type, old_value, new_value)
VALUES (2, 'delivery_order', 1, 'UPDATE',
      '{"order_status": "ASSIGNED"}'::jsonb,
      '{"order_status": "IN_TRANSIT"}'::jsonb);

SELECT entity_name, action_type, old_value->>'order_status' AS old_st,
       new_value->>'order_status' AS new_st
FROM   audit_log
WHERE  record_id = 1;
-- Expected: 1 row showing transition from ASSIGNED to IN_TRANSIT
```

Expected: PASS. Audit trail reliably captures actor and payload delta.

0.23. Query and Performance Tests

Performance tests were conducted using PostgreSQL EXPLAIN ANALYZE on representative datasets.

Table 6. Performance evaluation results

Query Type	Dataset Size	Filter / Condition	Exec. Time	Result
Tracking lookup	100,000 records	activity_id + order	< 4 ms	PASS
Order history	50,000 records	order_id + order	< 2 ms	PASS
Customer orders	10,000 orders	customer_id filter	< 3 ms	PASS
Station package events	25,000 events	package_id filter	< 2 ms	PASS
Audit log search	200,000 records	user_id + timestamp	< 8 ms	PASS

All queries utilized targeted index scans (idx_tracking_activity_time, idx_osh_order, idx_order_customer, idx_pse_package, idx_audit_time), confirming that query latency remains well beneath the 3-second operational threshold.

0.24. Chapter Summary

This chapter reported the comprehensive testing of the SmartDroneDelivery database implementation. Primary key, foreign key, unique, and check constraints were systematically evaluated across all twenty-three tables.

Business rule tests verified multiple customer addresses, drone concurrency separation, rescheduling tracking via `previous_activity_id`, package station pickup events, and unique delivery confirmations with digital proof keys. Append-only history preservation and JSON audit logging were verified. Finally, query execution evaluations confirmed that the indexing strategy delivers fast query response times under high data volumes.

Conclusion

0.25. Database Design Results

This document has presented the complete database design for the SmartDroneDelivery platform, covering the analysis of data requirements and business rules, the conceptual entity model, the logical relational schema, and the physical PostgreSQL implementation.

The final database consists of twenty-three tables organized into seven functional groups: user and authorization, customer and address, delivery order and package, landing station and operator management, delivery execution and telemetry, delivery confirmation, and audit, route, and AI recommendation tables. The design satisfies all thirty-nine business requirements and business rules identified in Chapter 0.4. The complete schema has been verified through constraint tests, business rule tests, history tests, query tests, and performance tests, all of which produced the expected results.

The most significant structural improvement introduced in this design is the replacement of a direct 1:1 relationship between a delivery order and a drone with the `delivery_activity` entity as an intermediary. This change enables the following capabilities:

- A single delivery order can be attempted multiple times. Each attempt is recorded as a separate `delivery_activity` row.
- When a delivery fails and is rescheduled, a new activity is created. The original failed activity and its exception and tracking records are preserved.
- The self-referencing foreign key `previous_activity_id` in `delivery_activity` maintains the unbroken lineage across consecutive rescheduling attempts.
- A drone can be assigned to multiple orders over time without storing redundant order information in the drone table.

- Tracking records and delivery exceptions are associated with a specific delivery activity, providing granular, per-attempt history.

Additional improvements include:

- Customer addresses are stored in a separate `customer_address` table, supporting multiple addresses per customer and enabling geographic coordinates to be stored as separate numeric columns for query and validation purposes.
- Station staffing and package handling are formalized through `station_operator_assignment` and `package_station_event`, enabling end-to-end verification of package pickup before drone departure.
- The `order_status_history` and `station_status_history` tables provide complete, append-only records of all status transitions. Business history and audit log data are maintained in separate tables serving distinct purposes.
- The `delivery_confirmation` table enforces UNIQUE constraints on both `order_id` and `activity_id`, implementing the business rule that each order has at most one confirmation and linking it directly to the successful attempt.
- The schema uses `evidence_object_key` to reference external object storage (e.g., MinIO/S3) for signatures and delivery photos, preventing binary bloat in PostgreSQL.
- All primary key columns use `BIGINT GENERATED ALWAYS AS IDENTITY` and all timestamp columns use `TIMESTAMPTZ`, in accordance with PostgreSQL best practices.
- The table named `system_user` avoids the reserved keyword `USER` in SQL.

0.26. Limitations

The following limitations are acknowledged in the current design:

- **No soft-delete mechanism:** The current schema does not implement soft deletion (e.g., an `is_deleted` flag). Deleted records cannot be recovered unless a backup is available. Future versions may add soft-delete support for critical entities.
- **Status values as check constraints:** Order, drone, station, and activity statuses are enforced through CHECK constraints on VARCHAR columns. If new status values are needed in the future, an ALTER TABLE statement is required. An alternative approach would be to use PostgreSQL ENUM types or a reference table, which would allow status values to be managed through INSERT statements rather than schema changes.
- **No partitioning:** The `tracking_record` table is expected to grow rapidly in production. The current design does not implement table partitioning. For production deployments at scale, range partitioning on `recorded_at` should be considered.

- **Performance tests on limited dataset:** Performance tests were conducted on a dataset of representative but limited size. Tests at production scale with millions of tracking records would require additional evaluation.
- **Optional tables not yet tested at scale:** The `delivery_route`, `route_point`, and `ai_recommendation` tables are included in the schema but were not the subject of dedicated performance tests in this document.

0.27. Future Database Improvements

Several improvements can be considered for future versions of the database design:

- **Table Partitioning:** Implement range partitioning on `tracking_record(recorded_at)` and `audit_log(action_time)` to manage data growth efficiently and improve query performance on time-bounded queries.
- **Status Reference Tables:** Replace CHECK constraints on status columns with dedicated reference tables, making status value management more flexible without requiring schema alterations.
- **Full-text Search:** Add PostgreSQL full-text search indexes on `delivery_exception.description` and `audit_log` fields to support operational analytics queries.
- **Database Triggers:** Implement database triggers to automatically insert rows into `order_status_history` and `station_status_history` whenever the corresponding status columns are updated, ensuring that history records are never omitted.
- **Materialized Views:** Create materialized views for common analytical queries, such as delivery success rates by station or average delivery duration by drone model, to reduce the overhead of repeated aggregation queries.
- **Connection Pooling and Read Replicas:** For production deployments, configure a connection pooler and read replicas to distribute query load and improve availability.
- **IoT Telemetry Integration:** Connect real-time telemetry from drone hardware directly into the `tracking_record` table through a dedicated ingestion pipeline, replacing or supplementing manual tracking updates.